

Shiv Nadar University Chennai
B.Tech Artificial Intelligence and Data Science
CS3807 – Deep Learning Laboratory

EXPERIMENT 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Dataset: CIFAR-10

Primary Architecture: VGG16

Name:	Prithviraj RP
Register Number:	24011101079
Semester:	V
Academic Year:	2026–27
Subject Code & Name:	CS3807 – Deep Learning Laboratory
Experiment:	4
GitHub:	https://github.com/Prithvi1247/Deep-Learning-Lab-Contents

Contents

1	Objective	2
2	Learning Outcomes	2
3	Tasks to be Performed	2
4	Background Theory	2
4.1	Evolution of CNN Architectures	2
4.2	LeNet-5	3
4.3	AlexNet	3
4.4	VGG16	3
4.5	GoogleNet (Inception Network)	3
4.6	ResNet	3
4.7	Comparison of Architectures	4
4.8	Dilated Convolution	4
4.9	Transpose Convolution	4
4.10	Transfer Learning	4
4.11	Fine-Tuning	5
5	CNN / Transfer Learning Architecture Used	5
6	Dataset Description	5
7	Experimental Procedure	5
7.1	Dataset Preparation	5
7.2	VGG16 Transfer Learning	5
7.3	Fine-Tuning	6
8	Source Code	6
9	Experimental Results	6
10	Mandatory Plots	7
10.1	Sample CIFAR-10 Images	7
10.2	Training and Validation Accuracy	7
10.3	Training and Validation Loss	8
10.4	Confusion Matrix	9
11	Performance Tables	9
11.1	Final Model Performance	9
11.2	Class-wise Classification Performance	10
11.3	Frozen Layers vs Fine-Tuning	10
11.4	Optimizer Comparison	10
11.5	CNN Architecture Comparison	11
11.6	Planned Hyperparameter Study	11

12 Discussion Questions	11
12.1 1. Why is AlexNet considered a breakthrough in deep learning?	11
12.2 2. Why does VGG16 use only 3×3 convolution filters?	12
12.3 3. Explain the advantages of the Inception module.	12
12.4 4. What is the purpose of residual learning?	12
12.5 5. Differentiate LeNet and ResNet.	12
12.6 6. What is Transfer Learning?	12
12.7 7. Why is fine-tuning required?	12
12.8 8. Explain the difference between dilated convolution and transpose convolution.	12
12.9 9. Why do pretrained models converge faster?	13
12.10 10. Compare the computational complexity of LeNet and ResNet.	13
13 Additional Exercises	13
14 Conclusion	13
15 References	14

1 Objective

The objectives of this experiment are to study the evolution of deep CNN architectures, implement transfer learning using a pretrained VGG16 model on the CIFAR-10 dataset, fine-tune selected convolutional layers, evaluate the model using standard classification metrics, and compare the performance of different CNN architectures (LeNet-5, AlexNet, VGG16 and ResNet50).

2 Learning Outcomes

After completing this experiment, the following outcomes are expected:

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine-tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3 Tasks to be Performed

1. Prepare and normalize the CIFAR-10 dataset.
2. Implement transfer learning using a pretrained VGG16 model.
3. Freeze the pretrained convolutional base and train a new classifier.
4. Fine-tune the last convolutional block and compare performance before and after fine-tuning.
5. Evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
6. Compare the performance of LeNet, AlexNet, VGG16 and ResNet50.
7. Compare Adam and SGD under the experimental training configuration.

4 Background Theory

4.1 Evolution of CNN Architectures

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulty. The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet and ResNet, each introducing important innovations.

Table 1: Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogLeNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

4.2 LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction. Its advantages include a small number of parameters, fast training, and suitability for OCR tasks.

4.3 AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training. Its major innovations were ReLU activation, Dropout, GPU implementation and data augmentation.

4.4 VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth. Its advantages are excellent feature extraction, a simple and uniform architecture, and high classification accuracy. In this experiment, the ImageNet-pretrained convolutional base is used without its original classification head. A Global Average Pooling layer, a dense ReLU layer and a ten-class Softmax output layer are added on top.

4.5 GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy et al. in 2014, introduced the Inception Module, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling, and the outputs are concatenated to obtain multi-scale feature representations. The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity. Its advantages include multi-scale feature extraction, fewer parameters than VGG16, high computational efficiency, and better classification performance.

4.6 ResNet

Increasing CNN depth often causes the vanishing gradient problem, making optimization difficult. ResNet solves this issue using skip (residual) connections, allowing gradients to

flow directly through the network. Instead of learning $H(x)$ directly, ResNet learns the residual mapping

$$F(x) = H(x) - x, \quad \text{so that} \quad \text{Output} = F(x) + x,$$

where $F(x)$ is the residual mapping and x is the identity mapping. Its advantages are the elimination of vanishing gradients, the ability to train very deep networks, faster convergence, and high ImageNet accuracy.

4.7 Comparison of Architectures

Table 2: Comparison of CNN Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

4.8 Dilated Convolution

Dilated (atrous) convolution increases the receptive field without increasing the number of parameters. The dilation rate determines the spacing between kernel elements: for a standard convolution $D = 1$, while for a dilated convolution $D > 1$, so that zeros are inserted between kernel elements. Applications include semantic segmentation, medical image analysis, satellite imagery and object localization.

4.9 Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling. Applications include image super-resolution, autoencoders, GANs, image generation and semantic segmentation.

4.10 Transfer Learning

Transfer learning uses a model pretrained on a large dataset such as ImageNet and adapts its learned feature representations to a new task. Instead of training a deep CNN completely from scratch, the pretrained convolutional layers are initially frozen and a new classification head is trained for the target dataset. The workflow used in this experiment is:

ImageNet pretrained model $\rightarrow$ remove classifier $\rightarrow$ freeze convolutional layers $\rightarrow$ add classifier $\rightarrow$ train

Transfer learning can provide faster convergence and good performance when the target dataset is relatively small, along with higher accuracy on small datasets and lower overall computational cost.

4.11 Fine-Tuning

After initially training the new classifier with the convolutional base frozen, the last convolutional block is unfrozen. The selected pretrained layers are then updated using a smaller learning rate so that the learned ImageNet features can adapt to CIFAR-10.

5 CNN / Transfer Learning Architecture Used

The classification head added on top of the pretrained VGG16 convolutional base was:

VGG16 (ImageNet weights) $\rightarrow$ Global Average Pooling $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dense(10, Softmax)

Since CIFAR-10 images are 32×32 , an internal resizing layer was used to resize them to 224×224 before passing them through VGG16.

6 Dataset Description

The experiment uses CIFAR-10. The dataset contains 50,000 training images and 10,000 testing images distributed across ten classes. Each image has dimensions $32 \times 32 \times 3$.

Table 3: CIFAR-10 Dataset

Property	Value
Training images	50,000
Testing images	10,000
Number of classes	10
Image dimensions	$32 \times 32 \times 3$
Classes	Airplane, Automobile, Bird, Cat, Deer Dog, Frog, Horse, Ship, Truck

The images were converted to floating-point values and normalized to the range $[0, 1]$.

7 Experimental Procedure

7.1 Dataset Preparation

The CIFAR-10 dataset was loaded using TensorFlow/Keras. Pixel values were normalized by dividing by 255.0. Ten representative images were displayed to verify the dataset and class labels.

7.2 VGG16 Transfer Learning

The VGG16 model was loaded with ImageNet-pretrained weights and without its original classifier. The convolutional base was frozen during the initial training stage, and the classification head described in Section 5 was trained on top of it. The initial training configuration was:

- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 32
- Epochs: 10
- Loss: Categorical Cross-Entropy

7.3 Fine-Tuning

For fine-tuning, the final VGG16 convolutional block was unfrozen while the earlier convolutional blocks remained frozen. The model was then trained for another five epochs using a learning rate of 0.0001.

8 Source Code

The complete implementation was developed using Python, TensorFlow/Keras, NumPy, Matplotlib and Scikit-learn. The notebook contains dataset loading and visualization, construction of the VGG16-based transfer-learning model, initial (frozen-base) training, fine-tuning of the final convolutional block, evaluation (accuracy, precision, recall, F1-score, confusion matrix), and the additional optimizer and architecture comparison experiments. The full source code is available on the GitHub repository linked on the title page.

9 Experimental Results

The VGG16-based transfer-learning model was trained on the CIFAR-10 dataset in two stages. In the first stage, the convolutional base was kept frozen and only the new classification head was trained for 10 epochs, reaching a test accuracy of 65.45%. In the second stage, the final convolutional block was unfrozen and the model was fine-tuned for 5 further epochs at a reduced learning rate, after which the training accuracy reached approximately 93.34%, the validation accuracy reached 83.70%, and the final test accuracy was 83.63%.

10 Mandatory Plots

10.1 Sample CIFAR-10 Images



Figure 1: Representative CIFAR-10 sample images with their class labels

Inference: The sample images demonstrate the ten object categories present in CIFAR-10. Since the images are only 32×32 pixels, several visually similar categories (for example cat and dog, or automobile and truck) can be difficult to distinguish even for a human observer.

10.2 Training and Validation Accuracy

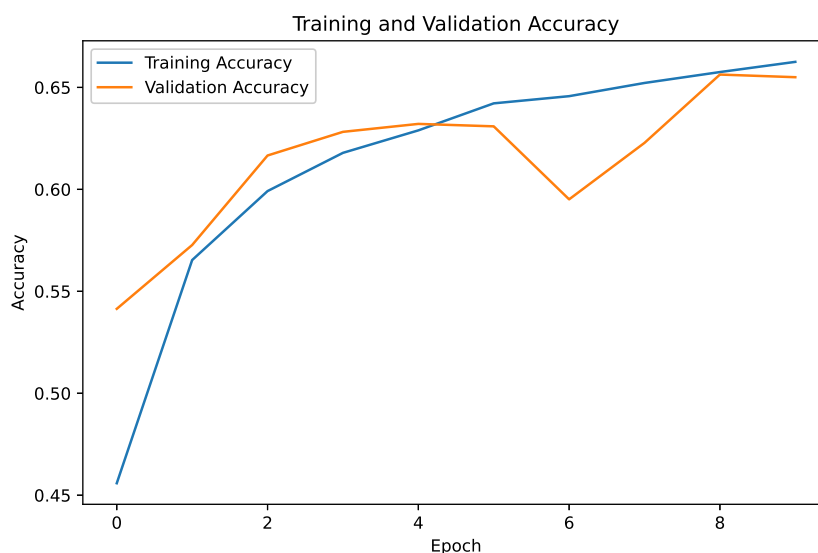


Figure 2: Training and validation accuracy during the initial frozen-base training stage

Inference: During the initial 10-epoch stage with the VGG16 convolutional base frozen, both training and validation accuracy rise steadily from around 45% to approximately 66%, with validation accuracy briefly dipping around epoch 6 before recovering. This

frozen-base stage corresponds to the 65.45% test accuracy reported before fine-tuning; the subsequent fine-tuning stage (Section 10) raised the training and validation accuracy substantially further.

10.3 Training and Validation Loss

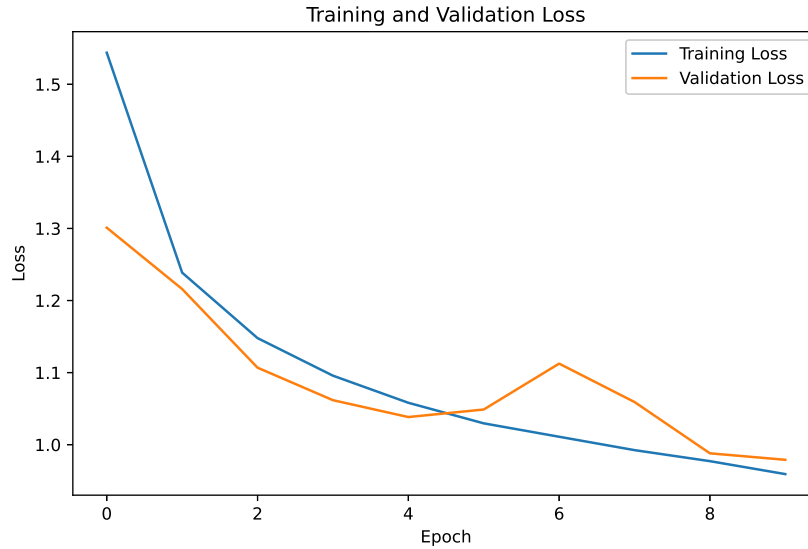


Figure 3: Training and validation loss during the initial frozen-base training stage

Inference: Both training and validation loss decrease steadily over the 10 epochs of the frozen-base stage, from around 1.5 to below 1.0. A small bump in validation loss appears around epoch 6, consistent with the corresponding dip in validation accuracy, before both curves resume their downward trend.

10.4 Confusion Matrix

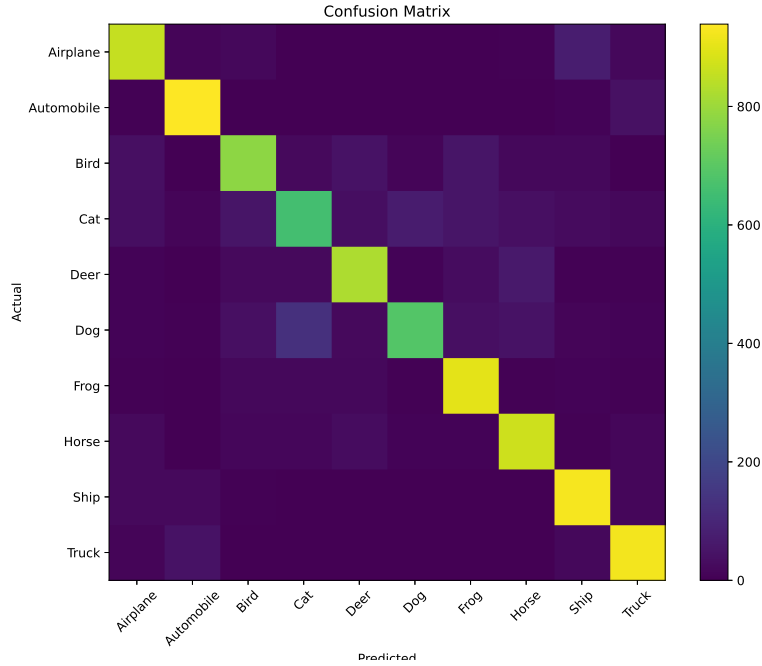


Figure 4: Confusion matrix for the fine-tuned VGG16 model on the CIFAR-10 test set

Inference: The diagonal elements represent correctly classified samples, while off-diagonal elements represent misclassifications. Classes such as automobile, frog, ship and truck show very strong diagonal values, while cat and dog show the most confusion with each other, consistent with their lower F1-scores in Table 8.

11 Performance Tables

11.1 Final Model Performance

Table 4: Final VGG16 Performance

Metric	Value
Training Accuracy	93.34%
Validation Accuracy	83.70%
Testing Accuracy	83.63%
Macro Precision	84%
Macro Recall	84%
Macro F1-score	83%
Total Parameters	14,848,586

11.2 Class-wise Classification Performance

Table 5: Fine-Tuned VGG16 Classification Report

Class	Precision	Recall	F1-score
Airplane	0.84	0.86	0.85
Automobile	0.90	0.94	0.92
Bird	0.81	0.78	0.80
Cat	0.75	0.66	0.70
Deer	0.84	0.82	0.83
Dog	0.87	0.69	0.77
Frog	0.83	0.90	0.86
Horse	0.83	0.87	0.85
Ship	0.83	0.93	0.87
Truck	0.87	0.92	0.89
Macro Average	0.84	0.84	0.83
Weighted Average	0.84	0.84	0.83

Inference: Automobile has the strongest class-wise performance with an F1-score of 0.92. Cat has the lowest F1-score of 0.70, while Dog has a relatively low recall of 0.69, indicating that these two classes are harder for the model to distinguish – consistent with the confusion matrix in Figure 5.

11.3 Frozen Layers vs Fine-Tuning

Table 6: Frozen Model vs Fine-Tuned Model

Model	Test Accuracy
Frozen convolutional base	65.45%
Fine-tuned final convolution block	83.63%
Improvement	18.18 percentage points

Inference: Fine-tuning produced a substantial improvement of 18.18 percentage points. This indicates that allowing the final convolutional block to adapt to CIFAR-10 made the pretrained ImageNet features more suitable for the target classification task.

11.4 Optimizer Comparison

Table 7: Adam vs SGD

Optimizer	Test Accuracy
Adam	31.82%
SGD	10.00%

Inference: Under this particular short training configuration (same classifier architecture, three epochs), Adam performed substantially better than SGD. The comparison should be interpreted within the chosen learning rate, batch size and epoch budget rather than as a general statement that Adam always outperforms SGD.

11.5 CNN Architecture Comparison

Table 8: CNN Architecture Comparison

Architecture	Parameters	Test Accuracy
LeNet-5	62,006	55.08%
AlexNet	3,302,986	64.05%
VGG16 (fine-tuned)	14,848,586	83.63%
ResNet50	23,721,610	32.32%

Inference: In this experimental run, the fine-tuned VGG16 achieved the highest test accuracy. AlexNet performed better than LeNet, while ResNet50 produced a lower accuracy under the short five-epoch transfer-learning configuration used for this additional experiment. The numerical comparison therefore reflects the specific training configurations used in this lab rather than the architectures' absolute capabilities.

11.6 Planned Hyperparameter Study

The lab manual additionally specifies the following hyperparameter grid for a broader study; within the time available for this experiment, the optimizer dimension of this grid (Adam vs SGD, Table 9) was executed in full, while the remaining dimensions are listed here for reference.

Table 9: Hyperparameter Study Design

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

12 Discussion Questions

12.1 1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet demonstrated that a deep convolutional neural network could achieve a major improvement in large-scale image classification. Its success on ImageNet helped establish

deep CNNs as a dominant approach for computer vision. The use of GPUs, ReLU activations, dropout and data augmentation also contributed to its success.

12.2 2. Why does VGG16 use only 3×3 convolution filters?

Small 3×3 filters allow VGG16 to build complex features through multiple convolutional layers while keeping the number of parameters relatively controlled. Stacking several small filters also introduces additional nonlinearities between layers.

12.3 3. Explain the advantages of the Inception module.

The Inception module processes information using multiple convolutional operations of different sizes and pooling in parallel. This allows the network to capture features at different spatial scales while using computational resources more efficiently.

12.4 4. What is the purpose of residual learning?

Residual learning makes very deep networks easier to optimize by learning a residual mapping instead of directly learning the complete desired transformation. Shortcut connections allow information and gradients to pass through the network more effectively.

12.5 5. Differentiate LeNet and ResNet.

LeNet is a relatively shallow CNN originally designed for handwritten digit recognition. ResNet is a much deeper architecture that uses residual/shortcut connections, allowing it to train effectively at substantially greater depth.

12.6 6. What is Transfer Learning?

Transfer learning is the reuse of knowledge learned by a model on one dataset or task for a different but related task. In this experiment, ImageNet-trained VGG16 features were adapted for CIFAR-10 classification.

12.7 7. Why is fine-tuning required?

The features learned from ImageNet are general but are not perfectly specialized for CIFAR-10. Fine-tuning selected pretrained layers allows the network to adapt these features to the target dataset and can improve classification performance.

12.8 8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution inserts gaps between convolution kernel elements to increase the receptive field without proportionally increasing the number of parameters or reducing spatial resolution. Transpose convolution performs learnable upsampling and increases the spatial dimensions of feature maps.

12.9 9. Why do pretrained models converge faster?

Pretrained models already contain useful low-level and mid-level visual representations. Therefore, the model starts optimization from meaningful features rather than random weights, reducing the amount of learning required for the target task.

12.10 10. Compare the computational complexity of LeNet and ResNet.

LeNet is much smaller and shallower, so it requires considerably fewer parameters and computations. ResNet is substantially deeper and computationally more expensive, although residual connections make training deep networks practical and effective.

13 Additional Exercises

1. **Transfer learning using VGG16:** Completed. Fine-tuned VGG16 achieved 83.63% test accuracy.
2. **Repeat using ResNet50:** Completed. The recorded test accuracy was 32.32% under the five-epoch configuration.
3. **Compare Adam and SGD:** Completed. Adam achieved 31.82% and SGD achieved 10.00% under the recorded three-epoch configuration.
4. **Frozen layers vs fine-tuning:** Completed. Accuracy improved from 65.45% to 83.63%.
5. **Precision, Recall and F1-score:** Completed. Macro precision was 0.84, macro recall was 0.84 and macro F1-score was 0.83.
6. **LeNet, AlexNet and ResNet comparison:** Completed. LeNet achieved 55.08%, AlexNet 64.05% and ResNet50 32.32%.

14 Conclusion

The transfer-learning experiment was successfully performed on the CIFAR-10 dataset using an ImageNet-pretrained VGG16 network. The convolutional base was initially frozen and a new classifier consisting of Global Average Pooling, a dense ReLU layer and a Softmax output layer was trained, reaching 65.45% test accuracy.

After unfreezing and fine-tuning the final convolutional block, the test accuracy increased to 83.63%, an improvement of 18.18 percentage points. The final model obtained a macro precision of 0.84, macro recall of 0.84 and macro F1-score of 0.83, with automobile as the best-classified category and cat and dog as the most challenging.

The additional experiments showed that, under the configurations used, AlexNet outperformed LeNet, while the fine-tuned VGG16 produced the strongest recorded classification result among all four architectures compared. The optimizer comparison also showed a higher test accuracy for Adam than for SGD under the short training configuration used. Overall, the experiment demonstrates the usefulness of pretrained CNN representations and fine-tuning for image classification, and illustrates how architectural depth, residual connections and training configuration each influence the final performance obtained.

15 References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, “Gradient-Based Learning Applied to Document Recognition,” Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” ICLR, 2015.
4. Christian Szegedy et al., “Going Deeper with Convolutions,” CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>